# VoucherHub — koncepcja systemu

Oct 20, 2026 · @Paweł

## O projekcie

VoucherHub to serwis, w którym partnerzy (siłownie, sauny, SPA) sprzedają vouchery na swoje usługi, a klienci je kupują i wykorzystują na miejscu. Działa jak bon na konkretną usługę: klient płaci z góry, dostaje kod i realizuje go u partnera.

To projekt studencki, więc opisujemy tylko podstawowe założenia i usługi. Płatność jest symulowana.

## Pojęcia

- **Oferta (szablon vouchera)** to opis usługi, np. „Wejście do sauny dla 2 osób, 89 zł, ważne 90 dni”. Tworzy ją partner, jedną dla wszystkich klientów.
- **Voucher (egzemplarz)** powstaje z oferty w chwili zakupu. Ma własny kod, klienta, datę zakupu, datę ważności i status. Każdy zakup to nowy voucher.

## Jak to działa

1. Partner dodaje ofertę, np. „Wejście do sauny dla 2 osób, 89 zł, ważne 90 dni”.
2. Klient przegląda oferty na stronie i kupuje wybraną.
3. Klient dostaje unikalny kod vouchera, np. `VH-7KQ2-M9XD-4TPA`.
4. Klient przychodzi do partnera i podaje kod.
5. Recepcja sprawdza kod w panelu partnera: na co jest voucher i czy jest ważny.
6. Recepcja realizuje voucher. Od tej chwili jest wykorzystany.

## Kto co może

**Klient** (bez zakładania konta):

- przegląda oferty,
- kupuje ofertę,
- sprawdza swój voucher po kodzie.

W wersji podstawowej kod vouchera jest jedynym dowodem zakupu, jak bilet na okaziciela. Klient widzi go tylko zaraz po zakupie i powinien go zapisać. E-mail podany przy zakupie zapisujemy, żeby w przyszłości można było dodać wysyłkę kodu na skrzynkę

klienta.

**Partner** (loguje się loginem i hasłem):

| Operacja | Oferta (szablon) | Voucher (egzemplarz) |
|---|---|---|
| Tworzenie | tak, dodaje nową | nie, powstaje tylko z zakupu przez klienta |
| Podgląd | tak, lista swoich ofert | tak, lista sprzedanych i sprawdzenie kodu |
| Edycja | nie (zmiana oznacza nową ofertę) | nie |
| Zmiana statusu | tak, wyłącza lub włącza sprzedaż | tylko realizacja: aktywny → wykorzystany |
| Usuwanie | nie (wystarczy wyłączenie) | nie |

Partner zarządza ofertami, ale nie voucherami. Voucher powstaje tylko z zakupu, a partner może go jedynie zrealizować.

Konta partnerów zakłada zespół projektu w danych startowych systemu. Nie ma rejestracji, wylogowania ani zmiany hasła. Po zalogowaniu partner dostaje token, który frontend dołącza do każdego zapytania. Token jest ważny 8 godzin, potem trzeba zalogować się ponownie. Przy każdym zapytaniu system sprawdza, czy partner jest nadal aktywny.

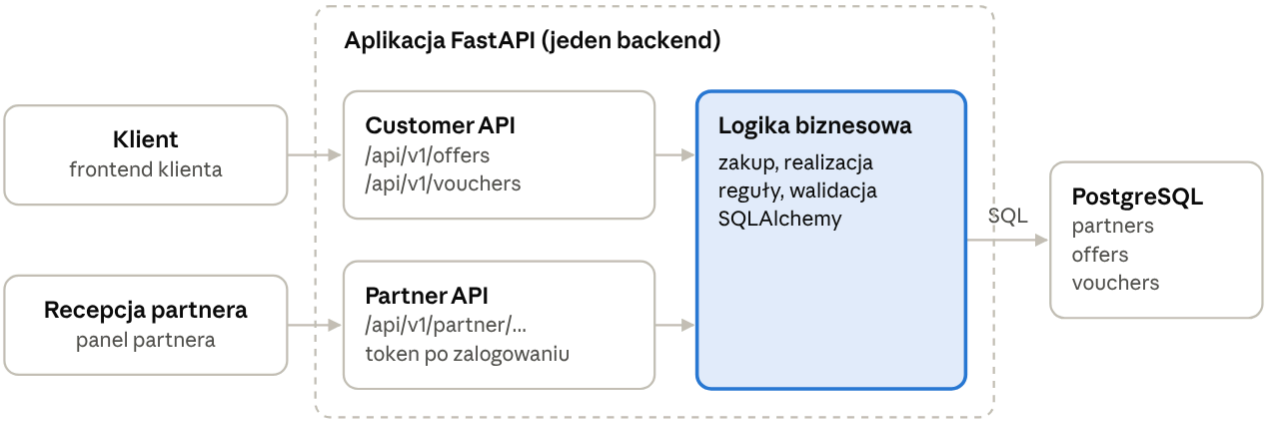
## Reguły biznesowe

1. Kupić można tylko aktywną ofertę aktywnego partnera.
2. Jeden zakup to jeden voucher.
3. Voucher jest ważny przez liczbę dni podaną w ofercie, licząc od zakupu.
4. Kod vouchera jest losowy i unikalny, nie da się go zgadnąć.
5. Voucher można zrealizować tylko raz.
6. Voucher realizuje tylko partner, który go sprzedał.
7. Po terminie ważności vouchera nie da się zrealizować.
8. Partner widzi i zmienia tylko swoje oferty oraz widzi tylko vouchery do swoich ofert.
9. Wyłączenie oferty nie unieważnia już kupionych voucherów.
10. Oferty nie można edytować. Żeby coś zmienić, partner wyłącza starą ofertę i dodaje nową.
11. Nieaktywny partner nie może się zalogować, a jego oferty znikają ze sprzedaży.

# Architektura

System składa się z frontendu (strona klienta i panel partnera), jednego backendu w Pythonie i bazy danych.

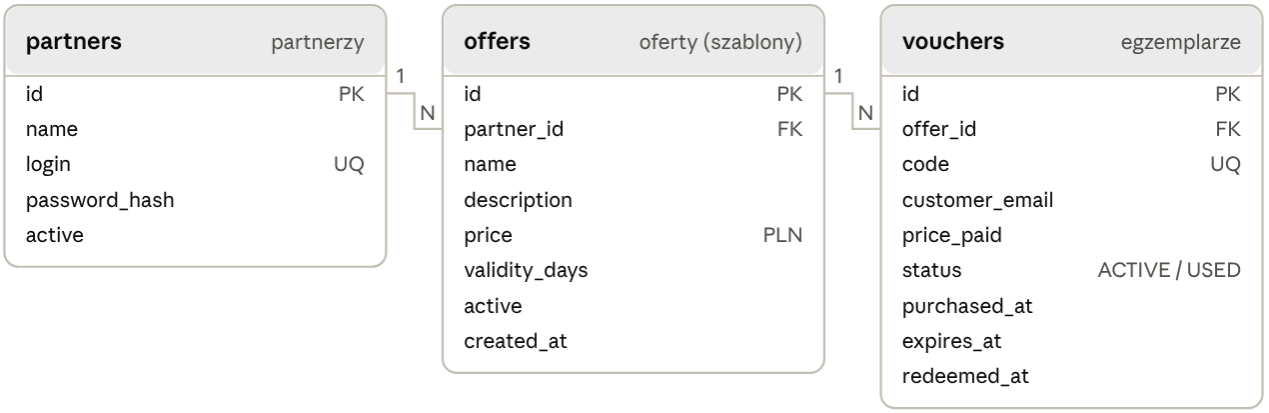
**Jedna aplikacja FastAPI obsługuje klientów i partnerów**




architektura · jeden backend, dwie grupy endpointów

# Dane

**Partner ma wiele ofert, oferta ma wiele voucherów**




PK klucz główny · FK klucz obcy · UQ wartość unikalna · 1–N jeden do wielu

schemat bazy danych · 3 tabele, 2 relacje 1:N

W bazie zapisujemy dwa statusy vouchera: `ACTIVE` (aktywny) i `USED` (wykorzystany). Status „wygasły” system wylicza z daty ważności (`expires_at`), więc nie trzeba go nigdzie przestawiać. Klient i recepcja widzą trzy statusy.

# Usługi API

Wzorujemy się na istniejących systemach voucherowych, np. Voucherify, które osobno sprawdzają kod i osobno go realizują.

| Kto | Usługa | Endpoint |
|---|---|---|
| Klient | Lista ofert | `GET /api/v1/offers` |
| Klient | Szczegóły oferty | `GET /api/v1/offers/{id}` |
| Klient | Zakup oferty | `POST /api/v1/offers/{id}/purchase` |
| Klient | Podgląd vouchera | `GET /api/v1/vouchers/{code}` |
| Partner | Logowanie (zwraca token) | `POST /api/v1/partner/login` |
| Partner | Lista moich ofert | `GET /api/v1/partner/offers` |
| Partner | Dodanie oferty | `POST /api/v1/partner/offers` |
| Partner | Wyłączenie lub włączenie oferty | `PATCH /api/v1/partner/offers/{id}` |
| Partner | Lista sprzedanych voucherów | `GET /api/v1/partner/vouchers` |
| Partner | Sprawdzenie kodu | `GET /api/v1/partner/vouchers/{code}` |
| Partner | Realizacja kodu | `POST /api/v1/partner/vouchers/{code}/redeem` |

Lista ofert, szczegóły oferty i podgląd vouchera pokazują klientowi nazwę partnera, u którego realizuje się voucher, oraz opis usługi.

Wszystkie usługi partnera poza logowaniem wymagają tokenu w nagłówku `Authorization: Bearer <token>`.

Gdy operacja się nie uda (np. voucher jest już wykorzystany albo przeterminowany), API zwraca kod błędu HTTP z opisem.

# Przebieg krok po kroku

Stan voucherów jest zapisany tylko w VoucherHub. Partner realizuje voucher przez panel partnera i nie prowadzi własnej ewidencji voucherów, więc o tym, czy realizacja się udała, decyduje jedno miejsce.

**1. Partner wystawia ofertę**

| Krok | Kto | Co się dzieje | Endpoint |
|---|---|---|---|
| 1 | Partner | Loguje się w panelu partnera i dostaje token | `POST /api/v1/partner/login` |
| 2 | Partner | Dodaje ofertę „Wejście do sauny dla 2 osób, 89 zł, ważne 90 dni” | `POST /api/v1/partner/offers` |
| 3 | Partner | Sprawdza, czy oferta jest na liście i aktywna | `GET /api/v1/partner/offers` |

### 2. Klient kupuje voucher

| Krok | Kto | Co się dzieje | Endpoint |
|---|---|---|---|
| 1 | Klient | Przegląda listę ofert z nazwami partnerów | `GET /api/v1/offers` |
| 2 | Klient | Otwiera szczegóły wybranej oferty | `GET /api/v1/offers/{id}` |
| 3 | Klient | Podaje e-mail i kupuje (płatność symulowana) | `POST /api/v1/offers/{id}/purchase` |
| 4 | System | Tworzy voucher z kodem i datą ważności, status: aktywny | (część kroku 3) |
| 5 | Klient | Zapisuje kod; później może sprawdzić voucher po kodzie | `GET /api/v1/vouchers/{code}` |

### 3. Klient realizuje voucher u partnera

| Krok | Kto | Co się dzieje | Endpoint |
|---|---|---|---|
| 1 | Klient | Przychodzi do sauny i podaje kod w recepcji | |
| 2 | Recepcja | Loguje się, jeśli token wygasł | `POST /api/v1/partner/login` |
| 3 | Recepcja | Wpisuje kod i widzi usługę, ważność i status | `GET /api/v1/partner/vouchers/{code}` |
| 4 | Recepcja | Klika „Zrealizuj” | `POST /api/v1/partner/vouchers/{code}/redeem` |

| Krok | Kto | Co się dzieje | Endpoint |
|---|---|---|---|
| 5 | System | Oznacza voucher jako wykorzystany i odpowiada „OK” | (część kroku 4) |
| 6 | Recepcja | Dopiero po „OK” wpuszcza klienta | |

Jeśli voucher jest już wykorzystany, wygasł albo należy do innego partnera, krok 4 kończy się błędem i recepcja nie wpuszcza klienta.

## Poza zakresem

Prawdziwe płatności, e-maile, PDF i kody QR, konta klientów, zwroty, cofanie realizacji, panel administratora.

## Technologie

- Backend: Python, FastAPI
- Baza danych: PostgreSQL
- Frontend: do ustalenia

## Pytania do prowadzącego

- Czy frontend ma być w określonej technologii?
- Czy klient musi mieć konto, czy wystarczy dostęp do vouchera po kodzie?
- Jakie diagramy są wymagane?
- Jakie są terminy kolejnych etapów?